

UNIVERSITY OF ST. THOMAS OF MOZAMBIQUE

Business School - Department of Computer Science

NOPIN REAL ESTATE APPS

HCI Final Assignment Report

A Dual-App Real Estate Discovery and Credit System Integration

Submitted By:

Dhalitso Albino Yotamo (Student ID: 202301014)

Course: 4th Year - Semester VII (Academic Year 2026)

Submitted To:

Marciano Ombe, Eng.

Course Module: Human-Computer Interaction (HCI)

Date of Submission: June 16, 2026

PART 1: SUPABASE VISITOR AUDITING & LOGGING

In the design of modern human-centered software, logging and auditing platform analytics is critical to understanding user behaviors, device constraints, and visual loading performance. For the Nopin Real Estate platform, we designed and implemented a silent background session tracker that logs visitor data directly into a Supabase database instance on page initialization and unload.

1.1 HCI and Usability Justification

Operating silently as a passive background process, this analytics module respects usability principles by introducing zero cognitive friction or extra UI elements. By automating geolocation and client hardware tracking, the system gains insight into the users' device characteristics (e.g., low-end mobile devices common in Mozambique) to guide future optimization of image sizes, avoiding intrinsic memory overload and map-loading delays on 3G/4G networks.

1.2 Audit Parameters Captured

- Device and Browser Specifications: Parses the Navigator User-Agent string to log the visitor's hardware specifications (e.g. mobile Safari, Android Chrome, phone model context).
- Geolocation Extraction: Fetches the public ipapi.co JSON API to dynamically identify the visitor's location (country, region, and city) on launch, which helps serve location-focused properties.
- Active Session Duration: Captures the entry time, runs a background counter that increments every 10 seconds, and writes the final duration value (in seconds) to the database upon page unload.

1.3 Supabase Integration and Testing

The system interacts with a PostgreSQL database on Supabase using the standard Supabase JavaScript Client SDK. The active hook (`useVisitorTracker.js`) listens for the browser's `visibilitychange` and `beforeunload` events to dispatch session logs securely. To verify the schema and validate analytic dashboards, we executed a seeder script (`seed_visitors.js`) that populates the database with 10 realistic Mozambican user sessions (from Maputo, Matola, Nampula, and Beira) indicating typical session times.

PART 2: SECURE OTP EMAIL AUTHENTICATION FLOW

A secure onboarding flow is essential to protect user profiles and reduce fake listing uploads. Due to SMTP configuration issues common with traditional mail server integrations, Nopin utilizes a secure client-side OTP (One-Time Password) verification flow powered directly by the EmailJS REST API.

2.1 Usability and Error Prevention (Nielsen's Heuristic #5)

Entering multi-digit codes on small mobile keyboards often leads to errors. Nopin employs an auto-focus shifting mechanism. When the user receives the OTP, they input the digits into 6 sequential input boxes. As each digit is typed, focus automatically shifts to the next input (and retreats on backspace). The 'Confirmar' button remains disabled until all 6 boxes are completed, preventing invalid code submissions.

2.2 Dynamic EmailJS Dispatch Protocol

1. Dynamic Code Generation: On submit, the React frontend generates a cryptographically secure 6-digit numeric string.
2. EmailJS API Request: The client dispatches a POST request to `api.emailjs.com/api/v1.0/email/send` containing the service ID, template ID, and public key. The parameters are passed dynamically:
 - `to_email`: The email input by the user.
 - `otp_code`: 'Ola! O seu codigo de verificacao temporario para o Nopin e: [code]'
3. Template Configuration: To reuse the template ID (`template_htfbto4`) for both login OTP and transaction receipts without text collision, the EmailJS dashboard template was cleaned to contain only the dynamic variable `{{otp_code}}` in its body. This allows the client application to format and deliver different styles of messages seamlessly.

2.3 Offline Testing Fallback

For offline testing or when environment credentials are not present in `.env`, the onboarding component catches the warning and triggers a high-fidelity visual toast notification containing the generated OTP. This permits continuous testing of the verification logic and session creation without blockages.

PART 3: EMOLUMENTS SHOPPING CART & CHECKOUT

Sellers who wish to promote their property listings (Premium tier costs 5 credits/day, Basic tier costs 1 credit/day) must top up their account balance via the Meu Perfil wallet. We implemented a shopping cart and checkout system to handle credit transactions securely.

3.1 Usability Justification and Conceptual Model

The top-up cart maps directly to the seller's conceptual model. It displays three predefined packages (Starter, Standard, Premium) alongside a custom quantity input. A live breakdown updates in real-time to show the quantity of credits, cost in Meticaís (1 credit = 10 MT), processing fees (0.00 MT), and the final payment total. This progressive cost disclosure removes ambiguity and prevents cart abandonment.

3.2 Logged-In User Email Synchronization

To guarantee receipt delivery and secure accounting, the checkout form dynamically extracts the email credentials of the logged-in user (user.email). We added a disabled, pre-filled E-mail de Faturação input field in the checkout step. This visually proves to the user that the platform is pulling their login credentials directly and will send the invoice receipt to the email they used to authenticate, enforcing data consistency and security.

3.3 Mobile Phone Number Normalization

Mozambican mobile users frequently input their numbers in different formats (e.g. +258 84 123 4567, 258 841234567, or 841234567). To prevent payment errors, we refactored the checkout handler to:

1. Remove all spaces and special symbols.
2. Strip the leading '+' if present.
3. Automatically check the length; if it is 9 digits and starts with 8, prepend the country code '258'.
4. Validate the normalized string against the standard expression: `/^2588[2-9]\d{7}$/`. This ensures standard inputs are automatically accepted and processed without error prompts.

PART 4: M-PESA API GATEWAY INTEGRATION & PROTOCOL

Integrating a mobile payment system requires strict adherence to security protocols, CORS, and cryptography. We built a secure proxy serverless function (`/api/mpesa.js`) that handles authentication and transaction requests with the official M-Pesa Mozambique sandbox API.

4.1 Ecosystem-Based Interaction (Norman's Feedback Loop)

To ensure a secure transaction flow, the mock in-app PIN pads were removed. The client application delegates the PIN input to the external mobile carrier's M-Pesa ecosystem. Tapping 'Pagar' triggers the serverless proxy, and the app interface enters a waiting state displaying 'Aguardando PIN no telemovel...'. The customer enters their PIN on their physical phone via the M-Pesa USSD STK Push popup, creating a reliable feedback loop and a secure checkout experience.

4.2 M-Pesa Proxy Protocol Steps

1. PEM Formatting Normalization: The function normalizes the `MPESA_PUBLIC_KEY` environment variable. It replaces literal `\n` and `'\n'` strings with actual newlines and wraps the key in public headers if missing, preventing runtime RSA decryption crashes.
2. RSA Encryption: The API Key is encrypted using the public key with RSA OAEP padding and converted to Base64.
3. Session Token: Sends a GET request to the M-Pesa sandbox Host (`api.sandbox.vm.co.mz`) with the Origin header set to `developer.mpesa.vm.co.mz` and Authorization header bearer token to generate a Session ID.
4. C2B Payment: Dispatches a POST request to the `singleStage` endpoint containing the amount, phone, and service provider code (171717). It blocks synchronously, waiting for the user to complete the PIN entry. On success, it returns `output_TransactionID` to credit the user's wallet.

4.3 Configured Sandbox Environment Credentials

The platform's local configuration has been updated with the official developer credentials:

- Host: `api.sandbox.vm.co.mz`
- Origin: `developer.mpesa.vm.co.mz`
- Service Provider Code: 171717
- API Key: `sm1s6d5q93uklcz94rf5sxhp0satgbq7`
 - Public Key: `MIICljANBgkqhkiG9w0BAQEFAAOCAg8AMIICGgKCAgEAmptSWqV7cGUU... (PEM encrypted)`